Reflection for C++26

Document #:	P2996R1
Date:	2023-12-18
Project:	Programming Language C++
Audience:	EWG

Reply-to: Wyatt Childers <wcc@edg.com>
Peter Dimov <pdimov@gmail.com>
Barry Revzin <barry.revzin@gmail.com>
Andrew Sutton <andrew.n.sutton@gmail.com>
Faisal Vali <faisalv@gmail.com>
Daveed Vandevoorde <daveed@edg.com>

Contents

1 Revision History

Since [P2996R0]:

- added links to Compiler Explorer demonstrating just about all of the examples
- respecified `synth_struct` to `define_class`
- respecified a few metafunctions to be functions instead of function templates
- introduced section on error handling mechanism and our preference for exceptions (removing invalid reflections)
- added ticket counter and variant examples
- collapsed `entity_ref` and `pointer_to_member` into `value_of`

2 Introduction

This is a proposal for a reduced initial set of features to support static reflection in C++. Specifically, we are mostly proposing a subset of features suggested in [P1240R2]:

- the representation of program elements via constant-expressions producing *reflection values* — *reflections* for short — of an opaque type `std::meta::info`,
- a *reflection operator* (prefix `^`) that produces a reflection value for its operand construct,
- a number of `consteval` *metafunctions* to work with reflections (including deriving other reflections), and
- constructs called *splicers* to produce grammatical elements from reflections (e.g., `[: refl :]`).

This proposal is not intended to be the end-game as far as reflection and compile-time metaprogramming are concerned. Instead, we expect it will be a useful core around which more powerful features will be added incrementally over time. In particular, we believe that most or all the remaining features explored in P1240R2 and that code injection (along the lines described in [P2237R0]) are desirable directions to pursue.

Our choice to start with something smaller is primarily motivated by the belief that that improves the chances of these facilities making it into the language sooner rather than later.

§ 2.1 Notable Additions to P1240

While we tried to select a useful subset of the P1240 features, we also made a few additions and changes. Most of those changes are minor. For example, we added a `std::meta::test_type` interface that makes it convenient to use existing standard type predicates (such as `is_class_v`) in reflection computations.

One addition does stand out, however: We have added metafunctions that permit the synthesis of simple struct and union types. While it is not nearly as powerful as generalized code injection (see [P2237R0]), it can be remarkably effective in practice.

§ 2.2 Why a single opaque reflection type?

Perhaps the most common suggestion made regarding the framework outlined in P1240 is to switch from the single `std::meta::info` type to a family of types covering various language elements (e.g., `std::meta::variable`, `std::meta::type`, etc.).

We believe that doing so would be a mistake with very serious consequences for the future of C++.

Specifically, it would codify the language design into the type system. We know from experience that it has been quasi-impossible to change the semantics of standard types once they were standardized, and there is no reason to think that such evolution would become easier in the future. Suppose for example that we had standardized a reflection type `std::meta::variable` in C++03 to represent what the standard called "variables" at the time. In C++11, the term "variable" was extended to include "references". Such an change would have been difficult to do given that C++ by then likely would have had plenty of code that depended on a type arrangement around the more restricted definition of "variable". That scenario is clearly backward-looking, but there is no reason to believe that similar changes might not be wanted in the future and we strongly believe that it behooves us to avoid adding undue constraints on the evolution of the language.

Other advantages of a single opaque type include:

— it makes no assumptions about the representation used within the implementation (e.g., it doesn't advantage one compiler over another),

— it is trivially extensible (no types need to be added to represent additional language elements and meta-elements as the language evolves), and

— it allows convenient collections of heterogeneous constructs without having to surface reference semantics (e.g., a `std::vector<std::meta::info>` can easily represent a mixed template argument list — containing types and nontypes — without fear of slicing values).

§ 2.3 Implementation Status

Lock3 implemented the equivalent of much that is proposed here in a fork of Clang (specifically, it worked with the P1240 proposal, but also included several other capabilities including a first-class injection mechanism).

EDG has an ongoing implementation of this proposal that is currently available on Compiler Explorer (thank you, Matt Godbolt). Nearly all of the examples below have links to compiler explorer demonstrating them.

Note that the implementation is not complete (notably, for debugging purposes, `name_of(^int)` yields an empty string and `name_of(^std::optional<std::string>)` yields `"optional"`, neither of which are what we want), and

does not include some of the other language features we would like to be able to take advantage of (like expansion statements) but will be regularly be updated as this paper progresses. Notably, it does not currently support expansion statements - and so a workaround that will be used in the linked implementations of examples is the following facility:

```
namespace __impl {
  template<auto... vals>
  struct replicator_type {
    template<typename F>
      constexpr void operator>>(F body) const {
        (body.template operator()<vals>(), ...);
      }
  };

  template<auto... vals>
  replicator_type<vals...> replicator = {};
}

template<typename R>
consteval auto expand(R range) {
  std::vector<std::meta::info> args;
  for (auto r : range) {
    args.push_back(reflect_value(r));
  }
  return substitute(^__impl::replicator, args);
}
```

Used like:

With expansion statements	With expand workaround

`template <typename E>`
`  requires std::is_enum_v<E>`
`constexpr std::string enum_to_string(E`
`        value) {`
`  template for (constexpr auto e :`
`        std::meta::members_of(^E)) {`
`    if (value == [:e:]) {`
`      return`
`        std::string(std::meta::name_of(e));`
`    }`
`  }`
 	
`  return "<unnamed>";`
`}`
	`template<typename E>`
	`  requires std::is_enum_v<E>`
	`constexpr std::string enum_to_string(E value) {`
	`  std::string result = "<unnamed>";`
	`  [:expand(std::meta::enumerators_of(^E)):] >>`
	`        [&]<auto e>{`
	`    if (value == [:e:]) {`
	`      result = std::meta::name_of(e);`
	`    }`
	`  };`
	`  return result;`
	`}`

§ 3 Examples

We start with a number of examples that show off what is possible with the proposed set of features. It is expected that these are mostly self-explanatory. Read ahead to the next sections for a more systematic description of each element of this proposal.

A number of our examples here show a few other language features that we hope to progress at the same time. This facility does not strictly rely on these features, and it is possible to do without them - but it would greatly help the usability experience if those could be adopted as well:

— expansion statements [P1306R1]

— non-transient constexpr allocation [P0784R7] [P1974R0] [P2670R1]

§ 3.1 Back-And-Forth

Our first example is not meant to be compelling but to show how to go back and forth between the reflection domain and the grammatical domain:

```
constexpr auto r = ^int;
typename[:r:] x = 42;       // Same as: int x = 42;
typename[:^char:] c = '*';  // Same as: char c = '*';
```

The `typename` prefix can be omitted in the same contexts as with dependent qualified names. For example:

```
using MyType = [:sizeof(int)<sizeof(long)? ^long : ^int:];  // Implicit "typename"
        prefix.
```

On Compiler Explorer.

§ 3.2 Selecting Members

Our second example enables selecting a member “by number” for a specific type:

```
struct S { unsigned i:2, j:6; };

consteval auto member_number(int n) {
  if (n == 0) return ^S::i;
  else if (n == 1) return ^S::j;
}

int main() {
  S s{0, 0};
  s.[:member_number(1):] = 42;  // Same as: s.j = 42;
  s.[:member_number(5):] = 0;   // Error (member_number(5) is not a constant).
}
```

This example also illustrates that bit fields are not beyond the reach of this proposal.

On Compiler Explorer

§ 3.3 List of Types to List of Sizes

Here, `sizes` will be a `std::array<std::size_t, 3>` initialized with `{sizeof(int), sizeof(float), sizeof(double)}`:

```
constexpr std::array types = {^int, ^float, ^double};
constexpr std::array sizes = []{
  std::array<std::size_t, types.size()> r;
  std::ranges::transform(types, r.begin(), std::meta::size_of);
  return r;
}();
```

Compare this to the following type-based approach, which produces the same array `sizes`:

```
template<class...> struct list {};

using types = list<int, float, double>;

constexpr auto sizes = []<template<class...> class L, class... T>(L<T...>) {
```

```
    return std::array<std::size_t, sizeof...(T)>{{ sizeof(T)... }};
}(types{});
```

On Compiler Explorer.

§ 3.4 Implementing `make_integer_sequence`

We can provide a better implementation of `make_integer_sequence` than a hand-rolled approach using regular template metaprogramming (although standard libraries today rely on an intrinsic for this):

```
#include <utility>
#include <vector>

template<typename T>
consteval std::meta::info make_integer_seq_refl(T N) {
  std::vector args{^T};
  for (T k = 0; k < N; ++k) {
    args.push_back(std::meta::reflect_value(k));
  }
  return substitute(^std::integer_sequence, args);
}

template<typename T, T N>
  using make_integer_sequence = [:make_integer_seq_refl<T>(N):];
```

On Compiler Explorer.

§ 3.5 Getting Class Layout

```
struct member_descriptor
{
  std::size_t offset;
  std::size_t size;
};

// returns std::array<member_descriptor, N>
template <typename S>
consteval auto get_layout() {
  constexpr auto members = nonstatic_data_members_of(^S);
  std::array<member_descriptor, members.size()> layout;
  for (int i = 0; i < members.size(); ++i) {
      layout[i] = {.offset=offset_of(members[i]), .size=size_of(members[i])};
  }
  return layout;
}

struct X
{
    char a;
    int b;
    double c;
};

/*constexpr*/ auto Xd = get_layout<X>();

/*
where Xd would be std::array<member_descriptor, 3>{{
  { 0, 1 }, { 4, 4 }, { 8, 8 }
```

```
}}
*/
```

On Compiler Explorer.

§ 3.6 Enum to String

One of the most commonly requested facilities is to convert an enum value to a string (this example relies on expansion statements):

```
template <typename E>
  requires std::is_enum_v<E>
constexpr std::string enum_to_string(E value) {
  template for (constexpr auto e : std::meta::members_of(^E)) {
    if (value == [:e:]) {
      return std::string(std::meta::name_of(e));
    }
  }

  return "<unnamed>";
}

enum Color { red, green, blue };
static_assert(enum_to_string(Color::red) == "red");
static_assert(enum_to_string(Color(42)) == "<unnamed>");
```

We can also do the reverse in pretty much the same way:

```
template <typename E>
  requires std::is_enum_v<E>
constexpr std::optional<E> string_to_enum(std::string_view name) {
  template for (constexpr auto e : std::meta::members_of(^E)) {
    if (name == std::meta::name_of(e)) {
      return [:e:];
    }
  }

  return std::nullopt;
}
```

But we don't have to use expansion statements - we can also use algorithms. For instance, `enum_to_string` can also be implemented this way (this example relies on non-transient constexpr allocation):

```
template <typename E>
  requires std::is_enum_v<E>
constexpr std::string enum_to_string(E value) {
  constexpr auto enumerators =
    std::meta::members_of(^E)
    | std::views::transform([](std::meta::info e){
        return std::pair<E, std::string>(std::meta::value_of<E>(e),
         std::meta::name_of(e));
      })
    | std::ranges::to<std::map>();

  auto it = enumerators.find(value);
  if (it != enumerators.end()) {
    return it->second;
  } else {
    return "<unnamed>";
```

```
  }
}
```

Note that this last version has lower complexity: While the versions using an expansion statement use an expected O(N) number of comparisons to find the matching entry, a `std::map` achieves the same with O(log(N)) complexity (where N is the number of enumerator constants).

On Compiler Explorer.

§ 3.7 Parsing Command-Line Options

Our next example shows how a command-line option parser could work by automatically inferring flags based on member names. A real command-line parser would of course be more complex, this is just the beginning.

```
template<typename Opts>
auto parse_options(std::span<std::string_view const> args) -> Opts {
  Opts opts;
  template for (constexpr auto dm : nonstatic_data_members_of(^Opts)) {
    auto it = std::ranges::find_if(args,
      [](std::string_view arg){
        return arg.starts_with("--") && arg.substr(2) == name_of(dm);
      });

    if (it == args.end()) {
      // no option provided, use default
      continue;
    } else if (it + 1 == args.end()) {
      std::print(stderr, "Option {} is missing a value\n", *it);
      std::exit(EXIT_FAILURE);
    }

    using T = typename[:type_of(dm):];
    auto iss = std::ispanstream(it[1]);
    if (iss >> opts.[:dm:]; !iss) {
      std::print(stderr, "Failed to parse option {} into a {}\n", *it,
        display_name_of(^T));
      std::exit(EXIT_FAILURE);
    }
  }
  return opts;
}

struct MyOpts {
  std::string file_name = "input.txt";  // Option "--file_name <string>"
  int    count = 1;                     // Option "--count <int>"
};

int main(int argc, char *argv[]) {
  MyOpts opts = parse_options<MyOpts>(std::vector<std::string_view>(argv+1,
        argv+argc));
  // ...
}
```

This example is based on a presentation by Matúš Chochlík.

On Compiler Explorer.

§ 3.8 A Simple Tuple Type

```
#include <meta>

template<typename... Ts> struct Tuple {
  struct storage;

  static_assert(is_type(define_class(^storage, {nsdm_description(^Ts)...})));
  storage data;

  Tuple(): data{} {}
  Tuple(Ts const& ...vs): data{ vs... } {}
};

template<typename... Ts>
  struct std::tuple_size<Tuple<Ts...>>: public integral_constant<size_t, sizeof...
        (Ts)> {};

template<std::size_t I, typename... Ts>
  struct std::tuple_element<I, Tuple<Ts...>> {
    static constexpr std::array types = {^Ts...};
    using type = [: types[I] :];
  };

consteval std::meta::info get_nth_nsdm(std::meta::info r, std::size_t n) {
  return nonstatic_data_members_of(r)[n];
}

template<std::size_t I, typename... Ts>
  constexpr auto get(Tuple<Ts...> &t) noexcept -> std::tuple_element_t<I,
        Tuple<Ts...>>& {
    return t.data.[:get_nth_nsdm(^decltype(t.data), I):];
  }
// Similarly for other value categories...
```

This example uses a "magic" `std::meta::define_class` template along with member reflection through the `nonstatic_data_members_of` metafunction to implement a `std::tuple`-like type without the usual complex and costly template metaprogramming tricks that that involves when these facilities are not available. `define_class` takes a reflection for an incomplete class or union plus a vector of nonstatic data member descriptions, and completes the give class or union type to have the described members.

On Compiler Explorer.

§ 3.9 A Simple Variant Type

Similarly to how we can implement a tuple using `define_class` to create on the fly a type with one member for each `Ts...`, we can implement a variant that simply defines a `union` instead of a `struct`. One difference here is how the destructor of a `union` is currently defined:

```
union U1 {
  int i;
  char c;
};

union U2 {
  int i;
  std::string s;
};
```

`U1` has a trivial destructor, but `U2`'s destructor is defined as deleted (because `std::string` has a non-trivial destructor). This is a problem because we need to define this thing… somehow. However, for the purposes of `define_class`,

there really is only one reasonable option to choose here:

```
template <class... Ts>
union U {
  // all of our members
  Ts... members;

  // a defaulted destructor if all of the types are trivially destructible
  constexpr ~U() requires (std::is_trivially_destructible_v<Ts> && ...) = default;

  // ... otherwise a destructor that does nothing
  constexpr ~U() { }
};
```

If we make `define_class` for a `union` have this behavior, then we can implement a `variant` in a much more straightforward way than in current implementations. This is not a complete implementation of `std::variant` (and cheats using libstdc++ internals, and also uses Boost.Mp11's `mp_with_index`) but should demonstrate the idea:

```
template <typename... Ts>
class Variant {
    union Storage;
    struct Empty { };

    static_assert(is_type(define_class(^Storage, {
        nsdm_description(^Empty, {.name="empty"}),
        nsdm_description(^Ts)...
    })));

    static constexpr std::array<std::meta::info, sizeof...(Ts)> types = {^Ts...};

    static consteval std::meta::info get_nth_nsdm(std::size_t n) {
        return nonstatic_data_members_of(^Storage)[n+1];
    }

    Storage storage_;
    int index_ = -1;

    // cheat: use libstdc++'s implementation
    template <typename T>
    static constexpr size_t accepted_index =
        std::__detail::__variant::__accepted_index<T, std::variant<Ts...>>;

    template <class F>
    constexpr auto with_index(F&& f) const -> decltype(auto) {
        return mp_with_index<sizeof...(Ts)>(index_, (F&&)f);
    }

public:
    constexpr Variant() requires std::is_default_constructible_v<[: types[0] :]>
        // should this work: storage_{. [: get_nth_nsdm(0) :]{} }
        : storage_{.empty={}}
        , index_(0)
    {
        std::construct_at(&storage_.[: get_nth_nsdm(0) :]);
    }

    constexpr ~Variant() requires (std::is_trivially_destructible_v<Ts> and ...) =
        default;
    constexpr ~Variant() {
        if (index_ != -1) {
            with_index([&](auto I){
```

```
                std::destroy_at(&storage_.[: get_nth_nsdm(I) :]);
            });
        }
    }

    template <typename T, size_t I = accepted_index<T&&>>
        requires (!std::is_base_of_v<Variant, std::decay_t<T>>)
    constexpr Variant(T&& t)
        : storage_{.empty={}}
        , index_(-1)
    {
        std::construct_at(&storage_.[: get_nth_nsdm(I) :], (T&&)t);
        index_ = (int)I;
    }

    // you can't actually express this constraint nicely until P2963
    constexpr Variant(Variant const&) requires (std::is_trivially_copyable_v<Ts> and
        ...) = default;
    constexpr Variant(Variant const& rhs)
            requires ((std::is_copy_constructible_v<Ts> and ...)
                and not (std::is_trivially_copyable_v<Ts> and ...))
        : storage_{.empty={}}
        , index_(-1)
    {
        rhs.with_index([&](auto I){
            constexpr auto nsdm = get_nth_nsdm(I);
            std::construct_at(&storage_.[: nsdm :], rhs.storage_.[: nsdm :]);
            index_ = I;
        });
    }

    constexpr auto index() const -> int { return index_; }

    template <class F>
    constexpr auto visit(F&& f) const -> decltype(auto) {
        if (index_ == -1) {
            throw std::bad_variant_access();
        }

        return mp_with_index<sizeof...(Ts)>(index_, [&](auto I) -> decltype(auto) {
            return std::invoke((F&&)f,  storage_.[: get_nth_nsdm(I) :]);
        });
    }
};
```

Effectively, `Variant<T, U>` synthesizes a union type `Storage` which looks like this:

```
union Storage {
    Empty empty;
    T unnamed_0;
    U unnamed_1;

    ~Storage() requires std::is_trivially_destructible_v<T> &&
        std::is_trivially_destructible_v<U> = default;
    ~Storage() { }
}
```

The question here is whether we should be should be able to directly initialize members of a defined union using a splicer, as in:

```
: storage{.[: get_nth_nsdm(0) :]={}}
```

Arguably, the answer should be yes - this would be consistent with how other accesses work.

On Compiler Explorer.

§ 3.10 Struct to Struct of Arrays

```
#include <meta>
#include <array>

template <typename T, std::size_t N>
struct struct_of_arrays_impl;

consteval auto make_struct_of_arrays(std::meta::info type,
                                     std::meta::info N) -> std::meta::info {
  std::vector<std::meta::info> old_members = nonstatic_data_members_of(type);
  std::vector<std::meta::info> new_members = {};
  for (std::meta::info member : old_members) {
    auto array_type = substitute(^std::array, {type_of(member), N });
    auto mem_descr = nsdm_description(array_type, {.name = name_of(member)});
    new_members.push_back(mem_descr);
  }
  return std::meta::define_class(
    substitute(^struct_of_arrays_impl, {type, N}),
    new_members);
}

template <typename T, size_t N>
using struct_of_arrays = [: make_struct_of_arrays(^T, ^N) :];
```

Example:

```
struct point {
  float x;
  float y;
  float z;
};

using points = struct_of_arrays<point, 30>;
// equivalent to:
// struct points {
//   std::array<float, 30> x;
//   std::array<float, 30> y;
//   std::array<float, 30> z;
// };
```

Again, the combination of `nonstatic_data_members_of` and `define_class` is put to good use.

On Compiler Explorer.

§ 3.11 Parsing Command-Line Options II

Now that we've seen a couple examples of using `std::meta::define_class` to create a type, we can create a more sophisticated command-line parser example.

This is the opening example for clap (Rust's **C**ommand **L**ine **A**rgument **P**arser):

```
struct Args : Clap {
  Option<std::string, {.use_short=true, .use_long=true}> name;
  Option<int, {.use_short=true, .use_long=true}> count = 1;
```

```
};

int main(int argc, char** argv) {
  auto opts = Args{}.parse(argc, argv);

  for (int i = 0; i < opts.count; ++i) {  // opts.count has type int
    std::print("Hello {}!", opts.name);   // opts.name has type std::string
  }
}
```

Which we can implement like this:

```
struct Flags {
  bool use_short;
  bool use_long;
};

// type that has a member optional<T> with some suitable constructors and members
template <typename T, Flags flags>
struct Option;

// convert a type (all of whose non-static data members are specializations of Option)
// to a type that is just the appropriate members.
// For example, if type is a reflection of the Args presented above, then this
// function would evaluate to a reflection of the type
// struct {
//   std::string name;
//   int count;
// }
consteval auto spec_to_opts(std::meta::info opts,
                            std::meta::info spec) -> std::meta::info {
  std::vector<std::meta::info> new_members;
  for (std::meta::info member : nonstatic_data_members_of(spec)) {
    auto new_type = template_arguments_of(type_of(member))[0];
    new_members.push_back(nsdm_description(new_type, {.name=name_of(member)}));
  }
  return define_class(opts, new_members);
}

struct Clap {
  template <typename Spec>
  auto parse(this Spec const& spec, int argc, char** argv) {
    std::vector<std::string_view> cmdline(argv+1, argv+argc)

    // check if cmdline contains --help, etc.

    struct Opts;
    static_assert(is_type(spec_to_opts(^Opts, ^Spec)));
    Opts opts;

    template for (constexpr auto [sm, om] :
        std::views::zip(nonstatic_data_members_of(^Spec),

        nonstatic_data_members_of(^Opts))) {
      auto const& cur = spec.[:sm:];
      constexpr auto type = type_of(om);

      // find the argument associated with this option
      auto it = std::ranges::find_if(cmdline,
        [&](std::string_view arg){
          return (cur.use_short && arg.size() == 2 && arg[0] == '-' && arg[1] ==
        name_of(sm)[0])
```

```
            || (cur.use_long && arg.starts_with("--") && arg.substr(2) ==
          name_of(sm));
        });

      // no such argument
      if (it == cmdline.end()) {
        if constexpr (has_template_arguments(type) and template_of(type) ==
        ^std::optional) {
          // the type is optional, so the argument is too
          continue;
        } else if (cur.initializer) {
          // the type isn't optional, but an initializer is provided, use that
          opts.[:om:] = *cur.initializer;
          continue;
        } else {
          std::print(stderr, "Missing required option {}\n", name_of(sm));
          std::exit(EXIT_FAILURE);
        }
      } else if (it + 1 == cmdline.end()) {
        std::print(stderr, "Option {} for {} is missing a value\n", *it, name_of(sm));
        std::exit(EXIT_FAILURE);
      }

      // found our argument, try to parse it
      auto iss = ispanstream(it[1]);
      if (iss >> opts.[:om:]; !iss) {
        std::print(stderr, "Failed to parse {:?} into option {} of type {}\n",
          it[1], name_of(sm), display_name_of(type));
        std::exit(EXIT_FAILURE);
      }
    }
    return opts;
  }
};
```

On Compiler Explorer.

§ 3.12 A Universal Formatter

This example is taken from Boost.Describe:

```
struct universal_formatter {
  constexpr auto parse(auto& ctx) { return ctx.begin(); }

  template <typename T>
  auto format(T const& t, auto& ctx) const {
    auto out = std::format_to(ctx.out(), "{}{{", name_of(^T));

    auto delim = [first=true]() mutable {
      if (!first) {
        *out++ = ',';
        *out++ = ' ';
      }
      first = false;
    };

    template for (constexpr auto base : bases_of(^T)) {
      delim();
      out = std::format_to(out, "{}", static_cast<[:base:] const&>(t));
    }
```

```
    template for (constexpr auto mem : nonstatic_data_members_of(^T)) {
      delim();
      out = std::format_to(out, ".{}={}", name_of(mem), t.[:mem:]);
    }

    *out++ = '}';
    return out;
  }
};

struct X { int m1 = 1; };
struct Y { int m2 = 2; };
class Z : public X, private Y { int m3 = 3; int m4 = 4; };

template <> struct std::formatter<X> : universal_formatter { };
template <> struct std::formatter<Y> : universal_formatter { };
template <> struct std::formatter<Z> : universal_formatter { };

int main() {
    std::println("{}", Z()); // Z{X{.m1 = 1}, Y{.m2 = 2}, .m3 = 3, .m4 = 4}
}
```

This example is not implemented on compiler explorer at this time, but only because of issues compiling both `std::format` and `fmt::format`.

§ 3.13 Implementing member-wise hash_append

Based on the [N3980] API:

```
template <typename H, typename T> requires std::is_standard_layout_v<T>
void hash_append(H& algo, T const& t) {
    template for (constexpr auto mem : nonstatic_data_members_of(^T)) {
        hash_append(algo, t.[:mem:]);
    }
}
```

§ 3.14 Converting a Struct to a Tuple

This approach requires allowing packs in structured bindings [P1061R5], but can also be written using `std::make_index_sequence`:

```
template <typename T>
constexpr auto struct_to_tuple(T const& t) {
  constexpr auto members = nonstatic_data_members_of(^T);

  constexpr auto indices = []{
    std::array<int, members.size()> indices;
    std::ranges::iota(indices, 0);
    return indices;
  }();

  constexpr auto [...Is] = indices;
  return std::make_tuple(t.[: members[Is] :]...);
}
```

An alternative approach is:

```
consteval auto struct_to_tuple_type(info type) -> info {
  return substitute(^std::tuple,
```

```
                    nonstatic_data_members_of(type)
                    | std::ranges::transform(std::meta::type_of)
                    | std::ranges::transform(std::meta::remove_cvref)
                    | std::ranges::to<std::vector>());
}

template <typename To, typename From, std::meta::info ... members>
constexpr auto struct_to_tuple_helper(From const& from) -> To {
  return To(from.[:members:]...);
}

template<typename From>
consteval auto get_struct_to_tuple_helper() {
  using To = [: struct_to_tuple_type(^From): ];

  std::vector args = {^To, ^From};
  for (auto mem : nonstatic_data_members_of(^From)) {
    args.push_back(reflect_value(mem));
  }

  /*
  Alternatively, with Ranges:
  args.append_range(
    nonstatic_data_members_of(^From)
    | std::views::transform(std::meta::reflect_value)
    );
  */

  return value_of<To(*)(From const&)>(
    substitute(^struct_to_tuple_helper, args));
}

template <typename From>
constexpr auto struct_to_tuple(From const& from) {
  return get_struct_to_tuple_helper<From>()(from);
}
```

Here, `struct_to_tuple_type` takes a reflection of a type like `struct { T t; U const& u; V v; }` and returns a reflection of the type `std::tuple<T, U, V>`. That gives us the return type. Then, `struct_to_tuple_helper` is a function template that does the actual conversion — which it can do by having all the reflections of the members as a non-type template parameter pack. This is a `constexpr` function and not a `consteval` function because in the general case the conversion is a run-time operation. However, determining the instance of `struct_to_tuple_helper` that is needed is a compile-time operation and has to be performed with a `consteval` function (because the function invokes `nonstatic_data_members_of`), hence the separate function template `get_struct_to_tuple_helper()`.

Everything is put together by using `substitute` to create the instantiation of `struct_to_tuple_helper` that we need, and a compile-time reference to that instance is obtained with `value_of`. Thus `f` is a function reference to the correct specialization of `struct_to_tuple_helper`, which we can simply invoke.

On Compiler Explorer, with a different implementation than either of the above.

§ 3.15 Compile-Time Ticket Counter

The features proposed here make it a little easier to update a ticket counter at compile time. This is not an ideal implementation (we'd prefer direct support for compile-time —— i.e., `consteval` — variables), but it shows how compile-time mutable state surfaces in new ways.

```
class TU_Ticket {
  template<int N> struct Helper {
    static constexpr int value = N;
  };
public:
  static consteval int next() {
    // Search for the next incomplete Helper<k>.
    std::meta::info r;
    for (int k = 0;; ++k) {
      r = substitute(^Helper, { std::meta::reflect_value(k) });
      if (is_incomplete_type(r)) break;
    }
    // Return the value of its member.  Calling static_data_members_of
    // triggers the instantiation (i.e., completion) of Helper<k>.
    return value_of<int>(static_data_members_of(r)[0]);
  }
};

int x = TU_Ticket::next();  // x initialized to 0.
int y = TU_Ticket::next();  // y initialized to 1.
int z = TU_Ticket::next();  // z initialized to 2.
```

Note that this relies on the fact that a call to `substitute` returns a specialization of a template, but doesn't trigger the instantiation of that specialization. Thus, the only instantiations of `TU_Ticket::Helper` occur because of the call to `nonstatic_data_members_of` (which is a singleton representing the lone `value` member).

On Compiler Explorer.

4 Proposed Features

4.1 The Reflection Operator (^)

The reflection operator produces a reflection value from a grammatical construct (its operand):

unary-expression:
…
^ ::
^ *namespace-name*
^ *type-id*
^ *cast-expression*

Note that *cast-expression* includes *id-expression*, which in turn can designate templates, member names, etc.

The current proposal requires that the *cast-expression* be:

— a *primary-expression* referring to a function or member function, or

— a *primary-expression* referring to a variable, static data member, or structured binding, or

— a *primary-expression* referring to a nonstatic data member, or

— a *primary-expression* referring to a template, or

— a constant-expression.

In a SFINAE context, a failure to substitute the operand of a reflection operator construct causes that construct to not evaluate to constant.

§ 4.2 Splicers ([:...:])

A reflection can be "spliced" into source code using one of several *splicer* forms:

— `[: r :]` produces an *expression* evaluating to the entity or constant value represented by `r`.

— `typename[: r :]` produces a *simple-type-specifier* corresponding to the type represented by `r`.

— `template[: r :]` produces a *template-name* corresponding to the template represented by `r`.

— `namespace[: r :]` produces a *namespace-name* corresponding to the namespace represented by `r`.

— `[:r:]::` produces a *nested-name-specifier* corresponding to the namespace, enumeration type, or class type represented by `r`.

The operand of a splicer is implicitly converted to a `std::meta::info` prvalue (i.e., if the operand expression has a class type that with a conversion function to convert to `std::meta::info`, splicing can still work.)

Attempting to splice a reflection value that does not meet the requirement of the splice is ill-formed. For example:

```
typename[: ^:: :] x = 0;  // Error.
```

(This proposal does not at this time propose range-based splicers as described in P1240. We still believe that those are desirable. However, they are more complex to implement and they involve syntactic choices that benefit from being considered along with other proposals that introduce pack-like constructs in non-template contexts. Meanwhile, we found that many very useful techniques are enabled with just the basic splicers presented here.)

§ 4.2.1 Range Splicers

The splicers described above all take a single object of type `std::meta::info` (described in more detail below). However, there are many cases where we don't have a single reflection, we have a range of reflections - and we want to splice them all in one go. For that, we need a different form of splicer: a range splicer.

Construct the struct-to-tuple example from above. It was demonstrates using a single splice, but it would be simpler if we had a range splice:

With Single Splice

```
template <typename T>
constexpr auto struct_to_tuple(T const& t) {
  constexpr auto members =
        nonstatic_data_members_of(^T);

  constexpr auto indices = []{
    std::array<int, members.size()> indices;
    std::ranges::iota(indices, 0);
    return indices;
  }();

  constexpr auto [...Is] = indices;
  return std::make_tuple(t.[: members[Is]
        :]...);
}
```

With Range Splice

```
template <typename T>
constexpr auto struct_to_tuple(T const& t) {
  constexpr auto members =
        nonstatic_data_members_of(^T);
  return std::make_tuple(t.[: ...members
        :]...);
}
```

A range splice, `[: ... r :]`, would accept as its argument a constant range of `meta::info`, `r`, and would behave as an unexpanded pack of splices. So the above expression

```
make_tuple(t.[: ... members :]...)
```

would evaluate as

```
make_tuple(t.[:members[0]:], t.[:members[1]:], ..., t.[:members[N-1]:])
```

This is a very useful facility indeed!

However, range splicing of dependent arguments is at least an order of magnitude harder to implement than ordinary splicing. We think that not including range splicing gives us a better chance of having reflection in C++26. Especially since, as this paper's examples demonstrate, a lot can be done without them.

Another way to work around a lack of range splicing would be to implement `with_size<N>(f)`, which would behave like `f(integral_constant<size_t, 0>{}, integral_constant<size_t, 0>{}, ..., integral_constant<size_t, N-1>{})`. Which is enough for a tolerable implementation:

```
template <typename T>
constexpr auto struct_to_tuple(T const& t) {
  constexpr auto members = nonstatic_data_members_of(^T);
  return with_size<members.size()>([&](auto... Is){
    return std::make_tuple(t.[: members[Is] :]...);
  });
}
```

§ 4.3 `std::meta::info`

The type `std::meta::info` can be defined as follows:

```
namespace std {
  namespace meta {
    using info = decltype(^int);
  }
}
```

In our initial proposal a value of type `std::meta::info` can represent:

— any (C++) type and type-alias

— any function or member function

— any variable, static data member, or structured binding

— any non-static data member

— any constant value

— any template

— any namespace

Notably absent at this time are general non-constant expressions (that aren't *expression-id*s referring to functions, variables or structured bindings). For example:

```
int x = 0;
void g() {
  [:^x:] = 42;      // Okay.  Same as: x = 42;
  x = [:^(2*x):];  // Error: "2*x" is a general non-constant expression.
  constexpr int N = 42;
  x = [:^(2*N):];  // Okay: "2*N" is a constant-expression.
}
```

Note that for `^(2*N)` an implementation only has to capture the constant value of `2*N` and not various other properties of the underlying expression (such as any temporaries it involves, etc.).

The type `std::meta::info` is a *scalar* type. Nontype template arguments of type `std::meta::info` are permitted. The entity being reflected can affect the linkage of a template instance involving a reflection. For example:

```
template<auto R> struct S {};

extern int x;
static int y;

S<^x> sx;  // S<^x> has external name linkage.
S<^y> sy;  // S<^y> has internal name linkage.
```

Namespace `std::meta` is associated with type `std::meta::info`: That allows the core meta functions to be invoked without explicit qualification. For example:

```
#include <meta>
struct S {};
std::string name2 = std::meta::name_of(^S);  // Okay.
std::string name1 = name_of(^S);             // Also okay.
```

4.4 Metafunctions

We propose a number of metafunctions declared in namespace `std::meta` to operator on reflection values. Adding metafunctions to an implementation is expected to be relatively “easy” compared to implementing the core language features described previously. However, despite offering a normal consteval C++ function interface, each on of these relies on “compiler magic” to a significant extent.

4.4.1 Error-Handling in Reflection

One important question we have to answer is: How do we handle errors in reflection metafunctions? For example, what does `std::meta::template_of(^int)` do? `^int` is a reflection of a type, but that type is not a specialization of a template, so there is no valid reflected template for us to return.

There are a few options available to us today:

1. This fails to be a constant expression (unspecified mechanism).
2. This returns an invalid reflection (similar to `NaN` for floating point) which carries source location info and some useful message. (This was the approach suggested in P1240.)
3. This returns `std::expected<std::meta::info, E>` for some reflection-specific error type `E` which carries source location info and some useful message (this could be just `info` but probably should not be).
4. This throws an exception of type `E` (which requires allowing exceptions to work during `constexpr` evaluation, such that an uncaught exception would fail to be a constant exception).

The immediate downside of (2), yielding a `NaN`-like reflection for `template_of(^int)` is what we do for those functions that need to return a range. That is, what does `template_arguments_of(^int)` return?

1. This fails to be a constant expression (unspecified mechanism).
2. This returns a `std::vector<std::meta::info>` containing one invalid reflection.
3. This returns a `std::expected<std::vector<std::meta::info>, E>`.
4. This throws an exception of type `E`.

Having range-based functions return a single invalid reflection would make for awkward error handling code. Using `std::expected` or exceptions for error handling allow for a consistent, more straightforward interface.

This becomes another situation where we need to decide an error handling mechanism between exceptions and not exceptions, although importantly in this context a lot of usual concerns about exceptions do not apply:

— there is no runtime (so concerns about runtime performance, object file size, etc. do not exist), and

— there is no runtime (so concerns about code evolving to add a new uncaught exception type do not apply)

There is one interesting example to consider to decide between `std::expected` and exceptions here:

```
template <typename T>
  requires (template_of(^T) == ^std::optional)
void foo();
```

If `template_of` returns an `excepted<info, E>`, then `foo<int>` is a substitution failure — `expected<T, E>` is equality-comparable to `T`, that comparison would evaluate to `false` but still be a constant expression.

If `template_of` returns `info` but throws an exception, then `foo<int>` would cause that exception to be uncaught, which would make the comparison not a constant expression. This actually makes the constraint ill-formed - not a substitution failure. In order to have `foo<int>` be a substitution failure, either the constraint would have to first check that `T` is a template or we would have to change the language rule that requires constraints to be constant expressions (we would of course still keep the requirement that the constraint is a `bool`).

The other thing to consider are compiler modes that disable exception support (like `-fno-exceptions` in GCC and Clang). Today, implementations reject using `try`, `catch`, or `throw` at all when such modes are enabled. With support for `constexpr` exceptions, implementations would have to come up with a strategy for how to support compile-time exceptions — probably by only allowing them in `consteval` functions (including `constexpr` function templates that were propagated to `consteval`).

Despite these concerns (and the requirement of a whole new language feature), we believe that exceptions will be the more user-friendly choice for error handling here, simply because exceptions are more ergonomic to use than `std::expected` (even if we adopt language features that make this type easier to use - like pattern matching and a control flow operator).

§ 4.4.2 Synopsis

Here is a synopsis for the proposed library API. The functions will be explained below.

```
namespace std::meta {
  // name and location
  consteval auto name_of(info r) -> string_view;
  consteval auto display_name_of(info r) -> string_view;
  consteval auto source_location_of(info r) -> source_location;

  // type queries
  consteval auto type_of(info r) -> info;
  consteval auto parent_of(info r) -> info;
  consteval auto dealias(info r) -> info;

  // template queries
  consteval auto template_of(info r) -> info;
  consteval auto template_arguments_of(info r) -> vector<info>;

  // member queries
  template<typename ...Fs>
```

```
  consteval auto members_of(info class_type, Fs ...filters) -> vector<info>;
template<typename ...Fs>
  consteval auto bases_of(info class_type, Fs ...filters) -> vector<info>;
consteval auto static_data_members_of(info class_type) -> vector<info>;
consteval auto nonstatic_data_members_of(info class_type) -> vector<info>;
consteval auto subobjects_of(info class_type) -> vector<info>;
consteval auto enumerators_of(info enum_type) -> vector<info>;

// substitute
consteval auto substitute(info templ, span<info const> args) -> info;

 // value_of
template<typename T>
  consteval auto value_of(info) -> T;

// test_type
consteval auto test_type(info templ, info type) -> bool;
consteval auto test_types(info templ, span<info const> types) -> bool;

// other_type_predicates
consteval auto is_public(info r) -> bool;
consteval auto is_protected(info r) -> bool;
consteval auto is_private(info r) -> bool;
consteval auto is_accessible(info r) -> bool;
consteval auto is_virtual(info r) -> bool;
consteval auto is_deleted(info entity) -> bool;
consteval auto is_defaulted(info entity) -> bool;
consteval auto is_explicit(info entity) -> bool;
consteval auto is_override(info entity) -> bool;
consteval auto is_pure_virtual(info entity) -> bool;
consteval auto is_bit_field(info entity) -> bool;
consteval auto has_static_storage_duration(info r) -> bool;
consteval auto is_nsdm(info entity) -> bool;
consteval auto is_base(info entity) -> bool;
consteval auto is_namespace(info entity) -> bool;
consteval auto is_function(info entity) -> bool;
consteval auto is_static(info entity) -> bool;
consteval auto is_variable(info entity) -> bool;
consteval auto is_type(info entity) -> bool;
consteval auto is_alias(info entity) -> bool;
consteval auto is_incomplete_type(info entity) -> bool;
consteval auto is_template(info entity) -> bool;
consteval auto is_function_template(info entity) -> bool;
consteval auto is_variable_template(info entity) -> bool;
consteval auto is_class_template(info entity) -> bool;
consteval auto is_alias_template(info entity) -> bool;
consteval auto has_template_arguments(info r) -> bool;
consteval auto is_constructor(info r) -> bool;
consteval auto is_destructor(info r) -> bool;
consteval auto is_special_member(info r) -> bool;

// reflect_value
template<typename T>
  consteval auto reflect_value(T value) -> info;

// define_class
struct nsdm_options_t;
consteval auto nsdm_description(info class_type, nsdm_options_t options = {}) ->
      info;
consteval auto define_class(info class_type, span<info const>) -> info;

// data_layout
```

```
  consteval auto offset_of(info entity) -> size_t;
  consteval auto size_of(info entity) -> size_t;
  consteval auto bit_offset_of(info entity) -> size_t;
  consteval auto bit_size_of(info entity) -> size_t;
  consteval auto alignment_of(info entity) -> size_t;
}
```

§ 4.4.3 `name_of`, `display_name_of`, `source_location_of`

```
namespace std::meta {
  consteval auto name_of(info r) -> string_view;
  consteval auto display_name_of(info r) -> string_view;
  consteval auto source_location_of(info r) -> source_location;
}
```

Given a reflection `r` that designates a declared entity `X`, `name_of(r)` returns a `string_view` holding the unqualified name of `X`. For all other reflections, an empty `string_view` is produced. For template instances, the name does not include the template argument list. The contents of the `string_view` consist of characters of the basic source character set only (an implementation can map other characters using universal character names).

Given a reflection `r`, `display_name_of(r)` returns a unspecified non-empty `string_view`. Implementations are encouraged to produce text that is helpful in identifying the reflected construct.

Given a reflection `r`, `source_location_of(r)` returns an unspecified `source_location`. Implementations are encouraged to produce the correct source location of the item designated by the reflection.

§ 4.4.4 `type_of`, `parent_of`, `dealias`

```
namespace std::meta {
  consteval auto type_of(info r) -> info;
  consteval auto parent_of(info r) -> info;
  consteval auto dealias(info r) -> info;
}
```

If `r` is a reflection designating a typed entity, `type_of(r)` is a reflection designating its type. If `r` is already a type, `type_of(r)` is not a constant expression. This can be used to implement the C `typeof` feature (which works on both types and expressions and strips qualifiers):

```
consteval auto do_typeof(std::meta::info r) -> std::meta::info {
  return remove_cvref(is_type(r) ? r : type_of(r));
}

#define typeof(e) [: do_typeof(^e) :]
```

If `r` designates a member of a class or namespace, `parent_of(r)` is a reflection designating its immediately enclosing class or namespace.

If `r` designates an alias, `dealias(r)` designates the underlying entity. Otherwise, `dealias(r)` produces `r`. `dealias` is recursive - it strips all aliases:

```
using X = int;
using Y = X;
static_assert(dealias(^int) == ^int);
static_assert(dealias(^X) == ^int);
static_assert(dealias(^Y) == ^int);
```

§ 4.4.5 `template_of`, `template_arguments_of`

```
namespace std::meta {
  consteval auto template_of(info r) -> info;
  consteval auto template_arguments_of(info r) -> vector<info>;
}
```

If `r` is a reflection designated a type that is a specialization of some template, then `template_of(r)` is a reflection of that template and `template_arguments_of(r)` is a vector of the reflections of the template arguments. In other words, the preconditions on both is that `has_template_arguments(r)` is `true`.

For example:

```
std::vector<int> v = {1, 2, 3};
static_assert(template_of(type_of(^v)) == ^std::vector);
static_assert(template_arguments_of(type_of(^v))[0] == ^int);
```

§ 4.4.6 `members_of`, `static_data_members_of`, `nonstatic_data_members_of`, `bases_of`, `enumerators_of`, `subobjects_of`

```
namespace std::meta {
  template<typename ...Fs>
    consteval auto members_of(info class_type, Fs ...filters) -> vector<info>;

  template<typename ...Fs>
    consteval auto bases_of(info class_type, Fs ...filters) -> vector<info>;

  consteval auto static_data_members_of(info class_type) -> vector<info> {
    return members_of(class_type, is_variable);
  }

  consteval auto nonstatic_data_members_of(info class_type) -> vector<info> {
    return members_of(class_type, is_nsdm);
  }

  consteval auto subobjects_of(info class_type) -> vector<info> {
    auto subobjects = bases_of(class_type);
    subobjects.append_range(nonstatic_data_members_of(class_type));
    return subobjects;
  }

  consteval auto enumerators_of(info enum_type) -> vector<info>;
}
```

The template `members_of` returns a vector of reflections representing the direct members of the class type represented by its first argument. Any nonstatic data members appear in declaration order within that vector. Anonymous unions appear as a nonstatic data member of corresponding union type. If any `Filters...` argument is specified, a member is dropped from the result if any filter applied to that members reflection returns `false`. E.g., `members_of(^C, std::meta::is_type)` will only return types nested in the definition of `C` and `members_of(^C, std::meta::is_type, std::meta::is_variable)` will return an empty vector since a member cannot be both a type and a variable.

The template `bases_of` returns the direct base classes of the class type represented by its first argument, in declaration order.

`enumerators_of` returns the enumerator constants of the indicated enumeration type in declaration order.

§ 4.4.7 `substitute`

```
namespace std::meta {
  consteval auto substitute(info templ, span<info const> args) -> info;
}
```

Given a reflection for a template and reflections for template arguments that match that template, `substitute` returns a reflection for the entity obtained by substituting the given arguments in the template. If the template is a concept template, the result is a reflection of a constant of type `bool`.

For example:

```
constexpr auto r = substitute(^std::vector, std::vector{^int});
using T = [:r:]; // Ok, T is std::vector<int>
```

This process might kick off instantiations outside the immediate context, which can lead to the program being ill-formed.

Note that the template is only substituted, not instantiated. For example:

```
template<typename T> struct S { typename T::X x; };

constexpr auto r = substitute(^S, std::vector{^int});  // Okay.
typename[:r:] si;  // Error: T::X is invalid for T = int.
```

4.4.8 `value_of<T>`

```
namespace std::meta {
  template<typename T> consteval auto value_of(info) -> T;
}
```

If `r` is a reflection for a constant-expression or a constant-valued entity of type `T`, `value_of<T>(r)` evaluates to that constant value.

If `r` is a reflection for a variable of non-reference type `T`, `value_of<T&>(r)` and `value_of<T const&>(r)` are lvalues referring to that variable. If the variable is usable in constant expressions [expr.const], `value_of<T>(r)` evaluates to its value.

If `r` is a reflection for a variable of reference type `T` usable in constant-expressions, `value_of<T>(r)` evaluates to that reference.

If `r` is a reflection of an enumerator constant of type `E`, `value_of<E>(r)` evaluates to the value of that enumerator.

If `r` is a reflection of a non-bit-field non-reference non-static member of type `M` in a class `C`, `value_of<M C::*>(r)` is the pointer-to-member value for that nonstatic member.

For other reflection values `r`, `value_of<T>(r)` is ill-formed.

The function template `value_of` may feel similar to splicers, but unlike splicers it does not require its operand to be a constant-expression itself. Also unlike splicers, it requires knowledge of the type associated with the entity reflected by its operand.

4.4.9 `test_type`, `test_types`

```
namespace std::meta {
  consteval auto test_type(info templ, info type) -> bool {
    return test_types(templ, {type});
  }
```

```
  consteval auto test_types(info templ, span<info const> types) -> bool {
    return value_of<bool>(substitute(templ, types));
  }
}
```

This utility translates existing metaprogramming predicates (expressed as constexpr variable templates or concept templates) to the reflection domain. For example:

```
struct S {};
static_assert(test_type(^std::is_class_v, ^S));
```

An implementation is permitted to recognize standard predicate templates and implement `test_type` without actually instantiating the predicate template. In fact, that is recommended practice.

§ 4.4.10 Other Singular Reflection Predicates

```
namespace std::meta {
  consteval auto is_public(info r) -> bool;
  consteval auto is_protected(info r) -> bool;
  consteval auto is_private(info r) -> bool;
  consteval auto is_accessible(info r) -> bool;
  consteval auto is_virtual(info r) -> bool;
  consteval auto is_deleted(info entity) -> bool;
  consteval auto is_defaulted(info entity) -> bool;
  consteval auto is_explicit(info entity) -> bool;
  consteval auto is_override(info entity) -> bool;
  consteval auto is_pure_virtual(info entity) -> bool;
  consteval auto is_bit_field(info entity) -> bool;
  consteval auto has_static_storage_duration(info r) -> bool;

  consteval auto is_nsdm(info entity) -> bool;
  consteval auto is_base(info entity) -> bool;
  consteval auto is_namespace(info entity) -> bool;
  consteval auto is_function(info entity) -> bool;
  consteval auto is_static(info entity) -> bool;
  consteval auto is_variable(info entity) -> bool;
  consteval auto is_type(info entity) -> bool;
  consteval auto is_alias(info entity) -> bool;
  consteval auto is_incomplete_type(info entity) -> bool;
  consteval auto is_template(info entity) -> bool;
  consteval auto is_function_template(info entity) -> bool;
  consteval auto is_variable_template(info entity) -> bool;
  consteval auto is_class_template(info entity) -> bool;
  consteval auto is_alias_template(info entity) -> bool;
  consteval auto has_template_arguments(info r) -> bool;
  consteval auto is_constructor(info r) -> bool;
  consteval auto is_destructor(info r) -> bool;
  consteval auto is_special_member(info r) -> bool;
}
```

§ 4.4.11 `reflect_value`

```
namespace std::meta {
  template<typename T> consteval auto reflect_value(T value) -> info;
}
```

This metafunction produces a reflection representing the constant value of the operand.

§ 4.4.12 `nsdm_description`, `define_class`

```
namespace std::meta {
  struct nsdm_options_t {
    optional<string_view> name;
    optional<int> alignment;
    optional<int> width;
  };
  consteval auto nsdm_description(info type, nsdm_options options = {}) -> info;
  consteval auto define_class(info class_type, span<info const>) -> info;
}
```

`nsdm_description` returns a reflection of a description of a non-static data member of given type. Optional alignment, bit-field-width, and name can be provided as well. If no `name` is provided, the name of the non-static data member is unspecified.

`define_class` takes the reflection of an incomplete class/struct/union type and a range of reflections of non-static data member descriptions and it completes the given class type with nonstatic data members as described (in the given order). The given reflection is returned. For now, only non-static data member reflections are supported (via `nsdm_description`) but the API takes in a range of `info` anticipating expanding this in the near future.

For example:

```
union U;
static_assert(is_type(define_class(^U, {
  nsdm_description(^int),
  nsdm_description(^char),
  nsdm_description(^double),
})));

// U is now defined to the equivalent of
// union U {
//   int _0;
//   char _1;
//   double _2;
// };

template<typename T> struct S;
constexpr auto U = define_class(^S<int>, {
  nsdm_description(^int, {.name="i", .align=64}),
  nsdm_description(^int, {.name="j", .align=64}),
});

// S<int> is now defined to the equivalent of
// template<> struct S<int> {
//   alignas(64) int i;
//   alignas(64) int j;
// };
```

When defining a `union`, if one of the alternatives has a non-trivial destructor, the defined union will *still* have a destructor provided - that simply does nothing. This allows implementing variant without having to further extend support in `define_class` for member functions.

§ 4.4.13 Data Layout Reflection

```
namespace std::meta {
  consteval auto offset_of(info entity) -> size_t;
  consteval auto size_of(info entity) -> size_t;

  consteval auto bit_offset_of(info entity) -> size_t;
  consteval auto bit_size_of(info entity) -> size_t;
```

```
  consteval auto alignment_of(info entity) -> size_t;
}
```

4.4.14 Other Type Traits

There is a question of whether all the type traits should be provided in `std::meta`. For instance, a few examples in this paper use `std::meta::remove_cvref(t)` as if that exists. Technically, the functionality isn't strictly necessary - since it can be provided indirectly:

Direct	Indirect
`std::meta::remove_cvref(type)`	`std::meta::substitute(^std::remove_cvref_t, {type})`

`std::meta::is_const(type)`	`std::meta::value_of<bool>`
	`  (std::meta::substitute(^std::is_const_v, {type}))`
	`std::meta::test_type(^std::is_const_v, type)`

Having `std::meta::meow` for every trait `std::meow` is more straightforward and will likely be faster to compile, though means we will have a much larger library API. There are quite a few traits in 21 [meta] - but it should be easy enough to specify all of them.

5 References

[N3980] H. Hinnant, V. Falco, J. Byteway. 2014-05-24. Types don't know #.
https://wg21.link/n3980

[P0784R7] Daveed Vandevoorde, Peter Dimov,Louis Dionne, Nina Ranns, Richard Smith, Daveed Vandevoorde. 2019-07-22. More constexpr containers.
https://wg21.link/p0784r7

[P1061R5] Barry Revzin, Jonathan Wakely. 2023-05-18. Structured Bindings can introduce a Pack.
https://wg21.link/p1061r5

[P1240R2] Daveed Vandevoorde, Wyatt Childers, Andrew Sutton, Faisal Vali. 2022-01-14. Scalable Reflection.
https://wg21.link/p1240r2

[P1306R1] Andrew Sutton, Sam Goodrick, Daveed Vandevoorde. 2019-01-21. Expansion statements.
https://wg21.link/p1306r1

[P1974R0] Jeff Snyder, Louis Dionne, Daveed Vandevoorde. 2020-05-15. Non-transient constexpr allocation using propconst.
https://wg21.link/p1974r0

[P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming.
https://wg21.link/p2237r0

[P2670R1] Barry Revzin. 2023-02-03. Non-transient constexpr allocation.
https://wg21.link/p2670r1

[P2996R0] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde. 2023-10-15. Reflection for C++26.

https://wg21.link/p2996r0